# Ten Ways to Use Agentic AI in Academic Research

Concrete uses of AI coding agents — Claude Code, Cursor, and similar — in academic research. For grad students, postdocs, and faculty; each item is something the workshop demonstrates on real artifacts: code, manuscripts, .bib files, slide decks.

What separates an agent from a chatbot is the working environment. Claude Code and its peers sit inside your project folder and can read every file in it — manuscript, data, codebook, .bib, slides, replication code, prior emails — at once. They can edit those files, run code, and inspect the result. The ten uses below are instances of the same underlying capability: one collaborator with your whole project loaded that can act, not just suggest. Working in ten separate chat windows loses most of this.

## 1. Literature triage and synthesis

Drop a stack of PDFs on the agent and ask it to extract each paper's core claim, method, and relevance to your project. The point is not to replace close reading; it's to drain the "maybe relevant" pile down to the papers you actually need to read, and to build a first-pass map of an unfamiliar literature before you commit to a deep dive. Useful for lit reviews, grant background sections, and the moment you realize you should have been reading some adjacent literature two years ago.

## 2. Learning new methods or unfamiliar literatures

When a statistical technique, archival approach, or body of scholarship outside your field becomes relevant, an agent will walk you through it with a worked example on your own data, point to canonical references, and flag the likely critics. Especially valuable for solo researchers, people between institutions, and anyone whose nearest peer group doesn't happen to cover the method they need this month. Best for the questions you'd never ask a colleague because they're too basic, or you've already asked twice.

## 3. Data wrangling and harmonization

Fuzzy-matching author names across datasets, OCR'ing scanned tables, deduping records, reconciling country codes between WDI and Penn World Table, assembling a panel from twenty messy raw files. The agent does not just produce plausible-looking code — it runs the merge, inspects what comes out, and iterates until the output passes the sanity checks you specify. This is where the productivity gap with the old StackOverflow workflow is largest.

## 4. Scrapers, APIs, and data pipelines

Pulling from Census, FRED, OpenAlex, Crossref, archival catalogs, or .gov sites that ship data as ugly HTML tables. The agent writes the request, parses the response, looks at what came back, and adjusts when the schema is not what the docs promised. For projects that need periodic refresh, it can also schedule the pipeline and diff successive runs so you notice when an upstream source quietly changes.

### 5. Writing, editing, and running empirical code

The full loop — pandas/R/Stata code written, run, error read, fixed, re-run — without you mediating every step. Most useful for the tedious-but-mechanical parts: reshaping panels, recoding values, building descriptives, formatting tables, and the matplotlib legend that refuses to stay where you put it. The difference from a chat-window workflow is that the agent does not stop at "here's what to try"; it tries, watches the output, and adjusts.

### 6. Verifying consistency and catching errors

Two angles on the same skill. First, code review — agents are unusually good at catching silent errors in pandas merges, off-by-one slicing, miscoded missing values, and the dropped observations you didn't notice. Second, cross-document consistency — numbers in the abstract matching numbers in Table 1, variable definitions matching the codebook, claims in the conclusion matching what the regressions actually show. The agent reads across files in a way a co-author skimming the PDF can't.

### 7. Research decision log and codebook

The highest-leverage habit on this list, and the one most users do not realize is possible. As the project evolves, the agent can maintain a README, codebook, and decision log — every variable construction, sample restriction, and specification choice recorded the moment it is made, with the reasoning attached. Reconstructing those decisions a year later from memory during the R&R is the part of empirical work that ages researchers fastest, and with an agent it is optional.

### 8. LaTeX and figure tooling

LaTeX has a learning curve measured in years and a long tail of obscure failure modes — table formatting to journal specs, Beamer, tikz, the compile error at 4am the morning of submission. Agents are fluent in both and do not tire of debugging it. The same goes for figures: ggplot, matplotlib, and the surprising fraction of writing time most academics lose to legend placement, axis labels, and color schemes, most of which now goes away.

### 9. Project upkeep

Decision logs record *why* you made empirical choices; project upkeep keeps the *artifacts* tidy. As a project ages, the .bib accumulates duplicates and broken entries, /scratch fills with notebooks no one will rerun, the README drifts from what the code actually does, and the folder layout that made sense at the start no longer does. An agent can periodically do all of this: clean the .bib and cross-check against Crossref or Zotero, archive stale notebooks, flag dead code paths, normalize file names, update the README. Useful before a submission, after a long project, or whenever the project starts to feel untidy in a way you can't quite name.

### 10. Teaching materials

Slides, problem sets, exam questions, TA guides, syllabi. The most concrete payoff: variants of the same problem so students cannot copy last year's solution set. Beyond that, agents adapt the same content across difficulty levels (PhD methods → MA methods → undergrad), draft worked solutions you can edit, and produce the marginal slide or worked example that turns a rough class into a polished one — especially when you are building a course from scratch.